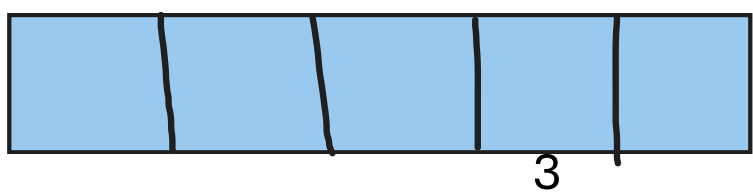


Lists

O(1)

Array ,



Arr[3] ::

FF00 + 3 * Sizeof (int)

Fixed Complixty : Access

Random Access

Arr
FF00

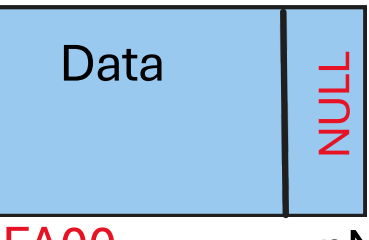
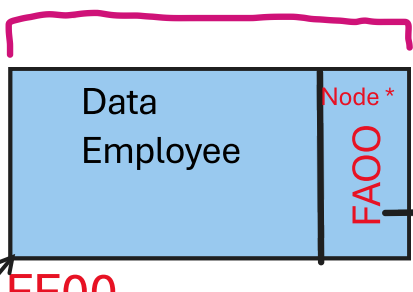
Fixed Size

```
Struct Employee
{
    int ID;
    short Age ;
    double Salary ;
    char Name[40]
};
Employee *Ptr ;
```

Ptr = new Employee();

```
Struct Node
{
    Employee Data;
    Node* pNext;
};
```

Node



20 - 100

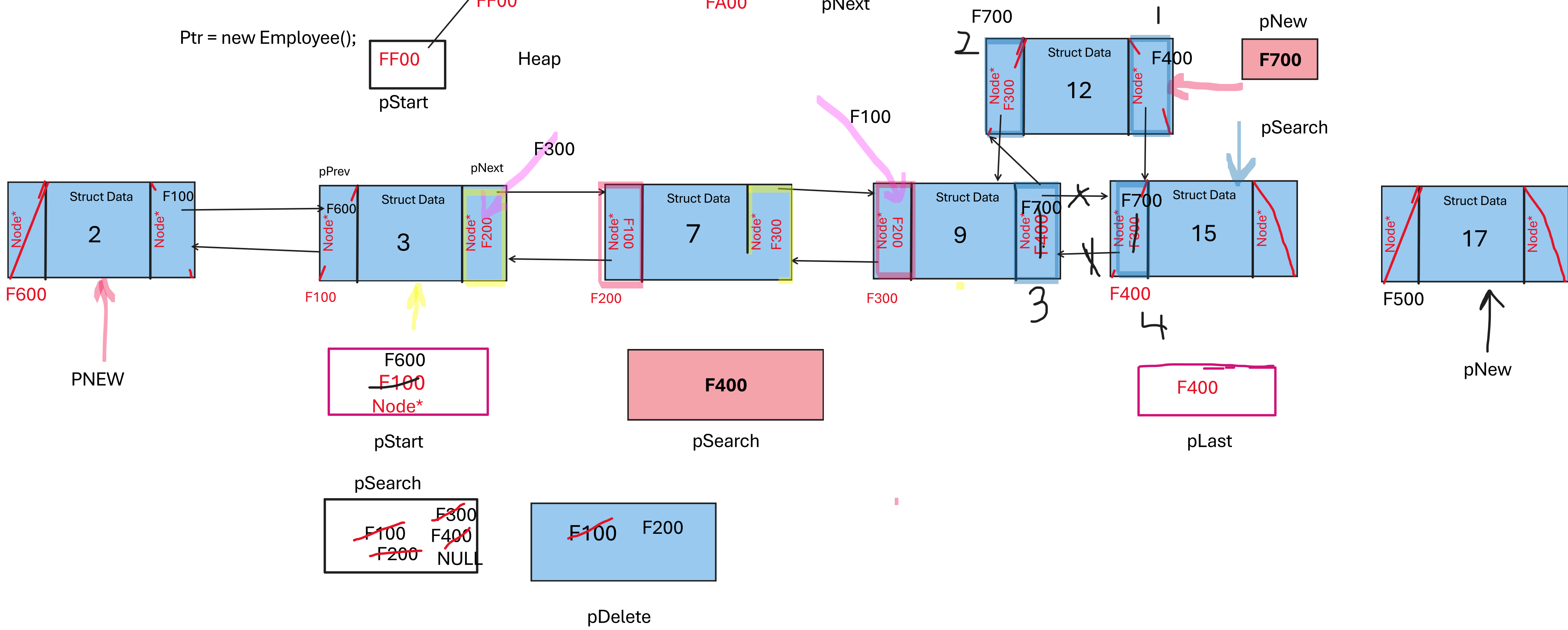
Array : 80 - 400

Double LL : 400 - 2000

Struct Node

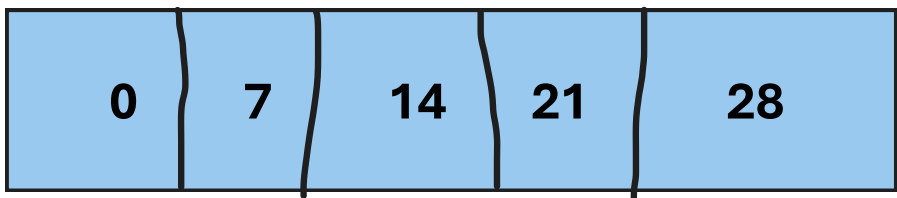
{ int Key ;

Node *pPrev , pNext ;



```
int* Ptr = new int [5];
```

```
for ( int i=0 ; i < 5 ; i++ )
Ptr[i] = 7 * i ;
```



F100



Ptr

```
delete []Ptr;
```

Class Node

```
{
    Employee Data;
    Node* pNext;
    Node* pPrev;
```

```
    Ctor(s);
    Destructor ();
}
```

class SortedDLL

```
{
    Node* pStart;
    Node* pLast;
```

```
    Ctor(s);
    Destructor()
    Copy Ctor();
```

```
    void InsertNode();
    bool DeleteNode(int ID);
    Node SearchList(int ID);
    bool DisplayNode(int ID);
    void DisplayALL();
    int NodeCount();
```

```
    operator[]
    operator =
```

```
}
```

Class StackDLL : public DLL

```
{
    void Push (Node pNew);
    Node Pop() ;
    Node Peek();
```

```
}
```

class Queue : public DLL

```
{
    void EnQ(Node pNew);
    Node Peek();
    Node DeQ();
```

```
}
```

class DLL

```
{
    Node* pStart;
    Node* pLast;
```

```
    Ctor(s);
    Destructor()
    Copy Ctor();
```

```
    void AddNode();
    bool DeleteNode(int ID);
    Node SearchList(int ID);
    bool DisplayNode(int ID);
    void DisplayALL();
    int NodeNumber ();
```

```
    operator[]
    operator =
```

```
}
```